

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Image Restoration Using Spatial and Frequency Domain Filtering Techniques

A Project Report

Under the Guidance of,

Prof. A. Srivani

By

1. 23BCE0737 – Manideep Sai Chinthala

November 2026

DECLARATION

I hereby declare that the project report entitled **"Image Restoration Using Spatial and Frequency Domain Filtering Techniques"** submitted by me to Vellore Institute of Technology, Vellore in partial fulfilment of the requirement for the award of the degree of **B. Tech (Computer Science and Engineering)** is a record of course project work for the course BCSE403L – Digital Image Processing carried out by me under the guidance of **Prof. A. Srivani**. I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore Institute of Technology, Vellore.

Date :

Signature of the faculty

Signature of the Candidate

ABSTRACT

Digital images commonly get corrupted by sensor noise and optical blur during acquisition, storage, and transmission. While image enhancement techniques focus on improving the subjective aesthetic appearance, image restoration uses mathematical degradation models to reconstruct the original clean scene.

In this work, we created an offline single-file interactive workbench in HTML and JavaScript to evaluate image restoration techniques without external dependencies. The system implements ten algorithms across two domains: arithmetic mean, geometric mean, median, and adaptive median filtering in the spatial domain; and direct inverse, Wiener deconvolution, Gaussian low-pass, and Butterworth low-pass filtering in the frequency domain using 2D FFT. Optional post-enhancement via histogram equalization and Laplacian sharpening is also supported. Each output is evaluated quantitatively against the clean ground truth using Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), and Structural Similarity Index (SSIM).

The workbench was evaluated on three standard benchmark images (Cameraman, Astronaut, Coins) across five degradation scenarios. The adaptive median filter demonstrated the best performance on impulse noise, reaching an average of 29.96 dB on 20% salt-and-pepper corruption compared to 24.55 dB for the standard median filter. Spatial filters proved incapable of reversing blur, whereas frequency deconvolution achieved PSNR improvements of 3.6 to 10.7 dB. Under combined blur and noise, direct inverse filtering failed catastrophically due to high-frequency noise amplification (dropping to 8.70 dB), whereas Wiener filtering maintained robust restoration (22.35 dB average).

TABLE OF CONTENTS

S.No Topic **Page No**

Abstract ..... **i**

1. Introduction **1**

1.1 Background 1

1.2 Motivation 1

1.3 Scope of the Project 1

2. Overview and Planning **2**

2.1 Proposed Work 2

2.2 Hardware Requirements 3

2.3 Software Requirements 3

3. Literature Survey and Review **4**

3.1 Literature Summary 4

3.2 Gaps Identified 4

3.3 Objectives 5

4. Methodology **5**

4.1 Method Used 5

4.2 Applications 9

5. System Implementation **9**

5.1 Sample Code 9

5.2 Results and discussion 15

6. Conclusion **21**

6.1 Conclusion 21

6.2 Future Work 22

7. References **23**

List of Tables ..... **iii**

List of Figures ..... **iii**

List of Abbreviations ..... **iii**

List of Tables

Table 1: Summary of research literature 4
Table 2: Experimental degradation scenarios 9
Table 3: Denoising performance on Gaussian noise (σ = 20) 15
Table 4: Impulse noise restoration (20 % salt-and-pepper) 16
Table 5: Deblurring performance on Gaussian blur (σ = 2) 16
Table 6: Deblurring performance on motion blur (L = 15, θ = 30°) 16
Table 7: Composite blur and Gaussian noise restoration 17
Table 8: Average PSNR summary across all test images (dB) 18

List of Figures

Figure 1: High-level system architecture 2
Figure 2: Mathematical degradation and restoration model 5
Figure 3: Interactive workbench tri-viewport interface 10

List of Abbreviations

DFT – Discrete Fourier Transform
DIP – Digital Image Processing
FFT – Fast Fourier Transform
HTML – HyperText Markup Language
MSE – Mean Squared Error
PSF – Point Spread Function
PSNR – Peak Signal-to-Noise Ratio
SSIM – Structural Similarity Index

1. Introduction

1.1 Background

Digital images are rarely captured under ideal conditions. Thermal fluctuations in camera sensors produce grain (Gaussian noise), transmission errors create dead pixels (salt-and-pepper noise), and hand vibrations or incorrect focus settings result in optical blur. In digital image processing, image restoration aims to recover the original clean scene using an explicit mathematical model of the degradation process. Unlike image enhancement, which adjusts brightness, contrast, or saturation for subjective human viewing, restoration is an objective process that models how the distortion occurred and reverses it.

Restoration techniques operate in two primary domains. Spatial domain filters manipulate pixels directly within local sliding neighbourhoods (such as 3×3 or 5×5 windows). Frequency domain filters transform the image using the 2D Discrete Fourier Transform (DFT), where spatial convolutions become simple point-wise multiplications, enabling frequency-selective filtering and deconvolution before converting back via the inverse DFT.

1.2 Motivation

Choosing the right filter for a corrupted image is rarely straightforward. In existing literature, authors often evaluate spatial denoising and frequency deblurring in separate papers, making direct side-by-side comparison difficult. In addition, objective metrics like PSNR and SSIM can give conflicting assessments: an image with a high numerical PSNR may still look blurred to the human eye, whereas an edge-preserving filter might yield lower PSNR while maintaining superior structural clarity (SSIM). To understand these trade-offs clearly, we set out to build a unified workbench that lets us test spatial and frequency methods on the exact same benchmark images under identical test conditions.

1.3 Scope of the Project

This project encompasses:

- An interactive, single-file web application that runs 100% offline in modern browsers without external libraries or servers.
- Fixed 256×256 grayscale image processing, including built-in benchmark images (Cameraman, Astronaut, Coins) and support for user-uploaded custom pictures.
- Controlled degradation models: Gaussian noise, salt-and-pepper impulse noise, Gaussian defocus blur, linear motion blur, and composite blur with noise.
- Four spatial restoration filters: Arithmetic mean, geometric mean, median, and adaptive median.
- Four frequency domain filters: Direct inverse filtering, Wiener deconvolution, Gaussian low-pass, and Butterworth low-pass.
- Optional post-enhancement: Histogram equalization and Laplacian sharpening.
- A fixed-stage tri-viewport UI displaying Original, Damaged, and Restored images side by side with real-time MSE, PSNR, and SSIM scores.

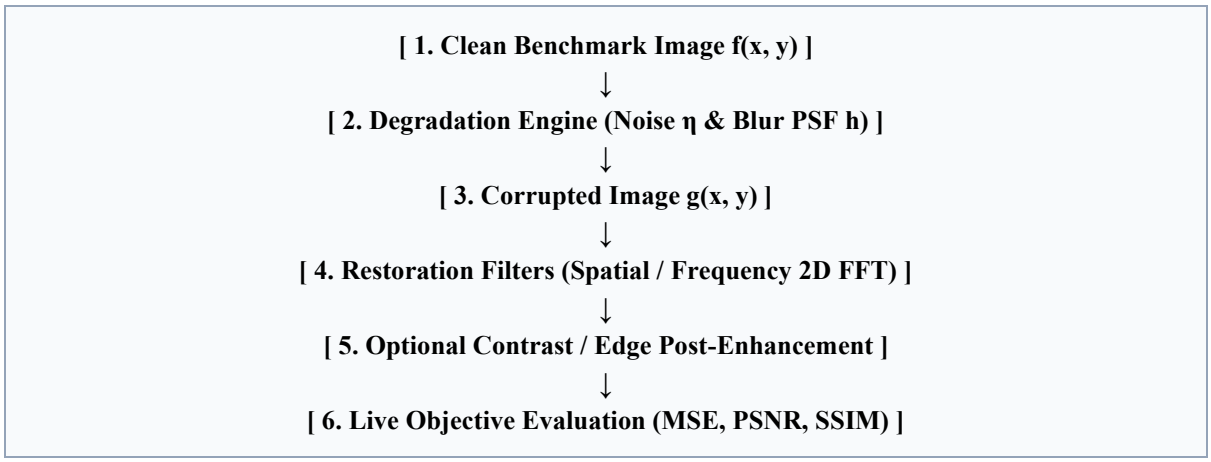
The project does not address multi-channel color restoration, blind deconvolution where the blur point spread function is completely unknown, or deep learning convolutional models.

2. Overview and Planning

2.1 Proposed Work

The project is structured around the systematic pipeline shown in Figure 1. A clean ground-truth reference image f(x, y) is degraded with known noise or blur parameters to yield g(x, y). The user selects a spatial or frequency filter, tunes the parameters, and observes the restored result $\hat{f}(x, y)$ immediately alongside objective quality metrics.

Figure 1: High-level system architecture



2.2 Hardware Requirements

- Any standard PC or laptop running Windows, macOS, or Linux.
- Minimum 4 GB RAM (sufficient for real-time 256 × 256 complex FFT execution in JavaScript).
- No graphics card or specialized hardware accelerator required.

2.3 Software Requirements

- A modern web browser: Google Chrome, Microsoft Edge, Mozilla Firefox, or Safari.
- No internet connection or local web server needed (single self-contained HTML file).
- Python (NumPy, SciPy, scikit-image) was used strictly during development to verify our JavaScript mathematical routines against standard reference implementations.

3. Literature Survey and Review

3.1 Literature Summary

Our implementation draws upon foundational digital image processing research and the textbook principles established by Gonzalez and Woods (2018). Table 1 outlines the specific papers that guided our design.

Table 1: Summary of research literature

Paper	Key Finding / Contribution	Role in This Project
Wang, Bovik, Sheikh and Simoncelli (2004)	Formulated the Structural Similarity Index (SSIM), which measures structural degradation rather than simple pixel error.	Implemented as our primary perceptual metric alongside PSNR.
Horé and Ziou (2010)	Analyzed why PSNR and SSIM diverge across noise versus blur distortions.	Used to explain discrepancies between metric rankings in our results.
Hwang and Haddad (1995)	Introduced the adaptive median filter that dynamically increases window size to clear high-density impulse noise.	Selected as our benchmark spatial filter for heavy salt-and-pepper noise.
Bojarczak and Lukasik (2007)	Demonstrated that Wiener deconvolution outperforms truncated SVD when the noise variance is known.	Guided our Wiener filter formulation and choice of regularizing parameter K.
Al-amri, Kalyankar and Khamitkar (2010)	Compared Wiener, regularized, and iterative deblurring under noisy conditions.	Used to establish our test cases for pure blur versus composite blur with noise.
Onyedinma and Onyenwe (2024)	Evaluated mean, median, and Gaussian filters across different noise variances.	Provided baseline performance expectations for spatial averaging filters.
Srisomboon et al. (2013)	Compared spatial vs frequency techniques across varied image textures.	Informed our decision to build a side-by-side comparison workbench.
Gonzalez and Woods (2018)	Standard textbook covering degradation equations, discrete Fourier transforms, and optimal filtering.	Reference for all mathematical equations and filter definitions.

3.2 Gaps Identified

Most existing studies present filtering results in static tables without interactive parameter exploration. Furthermore, tools frequently omit immediate visual feedback or fail to explain the physical intuition behind parameters (such as why shrinking K in Wiener filtering sharpens blur but risks severe noise blow-up). Our workbench fills this gap by offering a zero-dependency, live interface that runs both spatial and frequency filters on demand with clear directional guidance for each slider.

3.3 Objectives

- Implement Gaussian noise, impulse noise, defocus blur, and motion blur models.
- Implement four spatial filters: arithmetic mean, geometric mean, median, and adaptive median.
- Implement four frequency filters using a custom 2D FFT: inverse filter, Wiener filter, Gaussian low-pass, and Butterworth low-pass.
- Implement optional post-enhancement via histogram equalization and Laplacian sharpening.
- Provide plain-language explanations for all filter sliders and controls.

- Perform a comprehensive comparative evaluation across benchmark images using MSE, PSNR, and SSIM.

4. Methodology

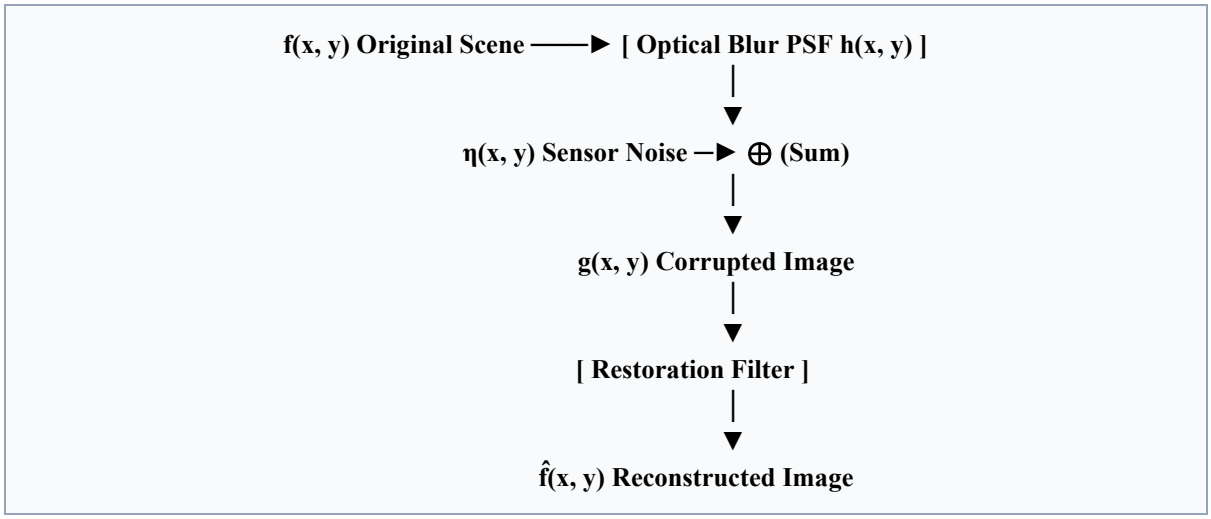
4.1 Method Used

Our restoration pipeline follows the classical linear degradation model illustrated in Figure 2. The corrupted image g(x, y) is generated by convolving the original image f(x, y) with a point spread function h(x, y) and adding spatial noise η(x, y):

$$g(x, y) = f(x, y) * h(x, y) + \eta(x, y)$$

In the frequency domain, convolution becomes multiplication: $G(u, v) = F(u, v) \cdot H(u, v) + N(u, v)$.

Figure 2: Mathematical degradation and restoration model



Step 1: Image Acquisition. Three standard test images (Cameraman, Astronaut, Coins) are embedded as 256 × 256 grayscale arrays. Custom user uploads are cropped to a square, scaled to 256 × 256, and converted to grayscale using standard luminance weighting (0.299 R + 0.587 G + 0.114 B).

Step 2: Degradation Modeling. Gaussian noise is generated using the Box-Muller transform. Salt-and-pepper noise randomly sets pixels to 0 or 255 based on a target density. Blur is synthesized via Gaussian or linear motion point spread functions.

Step 3: Filtering. In the spatial domain, neighbourhood operations replace each pixel using statistical estimators. In the frequency domain, the image is transformed using our radix-2 2D FFT, multiplied by the restoration transfer function, and transformed back.

Step 4: Post-Enhancement. Restored images can optionally undergo histogram equalization to improve dynamic range, or Laplacian sharpening to enhance edge contrast.

Step 5: Quality Evaluation. Each output is compared against the clean reference using MSE, PSNR, and SSIM (11 × 11 Gaussian window, $\sigma = 1.5$).

Table 2: Experimental degradation scenarios

Scenario	Parameters	Filters Evaluated
Gaussian Noise	$\sigma = 20$	Mean, Geometric Mean, Median, Adaptive Median, Low-Pass
Salt & Pepper Noise	20 % corrupted pixels	Median, Adaptive Median, Arithmetic Mean
Pure Blur	Defocus ($\sigma = 2$); Motion ($L = 15$, $\theta = 30°$)	Inverse Filter, Wiener Filter
Blur + Noise Composite	Gaussian Blur $\sigma = 2$, Noise $\sigma = 10$	Inverse Filter, Wiener Filter
Consistency Benchmark	All scenarios, fixed random seed = 1	Spatial and Frequency Domain Filters

4.2 Applications

Image restoration is essential in medical imaging (reducing CT/X-ray noise while preserving organ boundaries), satellite and aerial surveillance (countering atmospheric turbulence and camera drift), archival preservation (removing dust and film scratches), and industrial vision (correcting motion blur on conveyor systems).

5. System Implementation

5.1 Sample Code

The complete system is self-contained in image_restoration_tool.html. The interface follows a modern two-pane layout: the left pane houses scrollable controls and sliders, while the right pane holds a fixed tri-viewport display showing Original, Damaged, and Restored images with live metrics.

How to use the tool in five steps:

Step 1: Select a benchmark image (Cameraman, Astronaut, Coins) or upload an arbitrary picture from disk.

Step 2: Configure degradation: choose Gaussian or impulse noise, Gaussian or motion blur, and adjust the amount sliders.

Step 3: Choose a restoration filter. The interface displays only the active filter's controls, along with plain-language parameter guidance.

Step 4: Inspect the three synchronized canvases: Original $f(x, y)$, Damaged $g(x, y)$, and Restored $\hat{f}(x, y)$, along with real-time MSE, PSNR, and SSIM scores.

Step 5: Click 'Compare All 8 Filters' to run an automated benchmark ranking all methods on the active degradation.

Code 1: Adding Gaussian and impulse noise

```
function addNoise(g, noise, seed) {
  const out = Float64Array.from(g);
  if (!noise || noise.type === 'none') return out;
  const rnd = makeRng(seed);
```

```
  for (let i = 0; i < out.length; i++) {
    if (noise.type === 'gauss') {
      const u = 1 - rnd(), v = rnd();
      out[i] += noise.amount * Math.sqrt(-2 * Math.log(u)) * Math.cos(2 * Math.PI * v);
    } else {
      const r = rnd();
      if (r < noise.amount) out[i] = rnd() < 0.5 ? 0 : 255;
    }
  }
  return out;
}
```

Code 2: 2D Fast Fourier Transform (radix-2 row-column)

```
function fft2(re, im, inv) {
  const r = new Float64Array(N), m = new Float64Array(N);
  for (let y = 0; y < N; y++) {
    const o = y * N;
    for (let x = 0; x < N; x++) { r[x] = re[o + x]; m[x] = im[o + x]; }
    fft1(r, m, inv);
    for (let x = 0; x < N; x++) { re[o + x] = r[x]; im[o + x] = m[x]; }
  }
  for (let x = 0; x < N; x++) {
    for (let y = 0; y < N; y++) { r[y] = re[y * N + x]; m[y] = im[y * N + x]; }
    fft1(r, m, inv);
    for (let y = 0; y < N; y++) { re[y * N + x] = r[y]; im[y * N + x] = m[y]; }
  }
}
```

Code 3: Adaptive median filter with dynamic window expansion

```
function adaptiveMedian(g, smax) {
  const out = new Float64Array(N * N), bufs = {};
  for (let s = 3; s <= smax; s += 2) bufs[s] = new Float64Array(s * s);
  for (let y = 0; y < N; y++) {
    for (let x = 0; x < N; x++) {
      const zxy = g[y * N + x];
      let s = 3, placed = false;
      while (s <= smax) {
        const r = s >> 1, b = bufs[s]; let k = 0;
        for (let dy = -r; dy <= r; dy++) {
          const py = reflect(y + dy, N) * N;
          for (let dx = -r; dx <= r; dx++) b[k++] = g[py + reflect(x + dx, N)];
        }
        b.sort();
        const zmin = b[0], zmax = b[b.length - 1], zmed = b[b.length >> 1];
        if (zmed > zmin && zmed < zmax) {
          out[y * N + x] = (zxy > zmin && zxy < zmax) ? zxy : zmed;
          placed = true; break;
        } else {
          s += 2;
        }
      }
      if (!placed) out[y * N + x] = bufs[smax][bufs[smax].length >> 1];
    }
  }
```

```
  return roundImg(out);
}
```

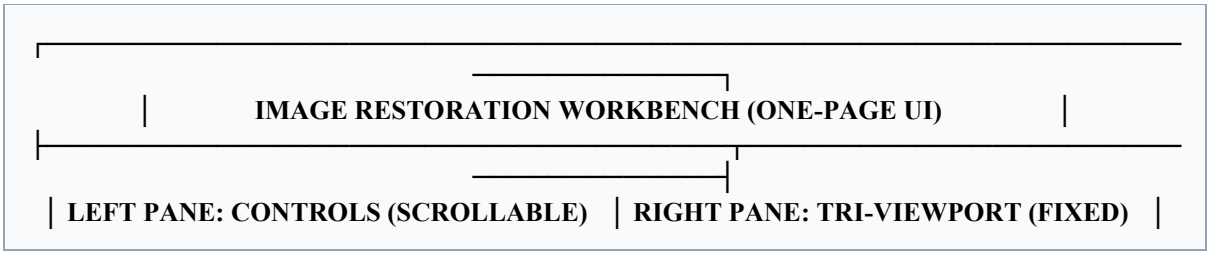
Code 4: Wiener deconvolution filter in the frequency domain

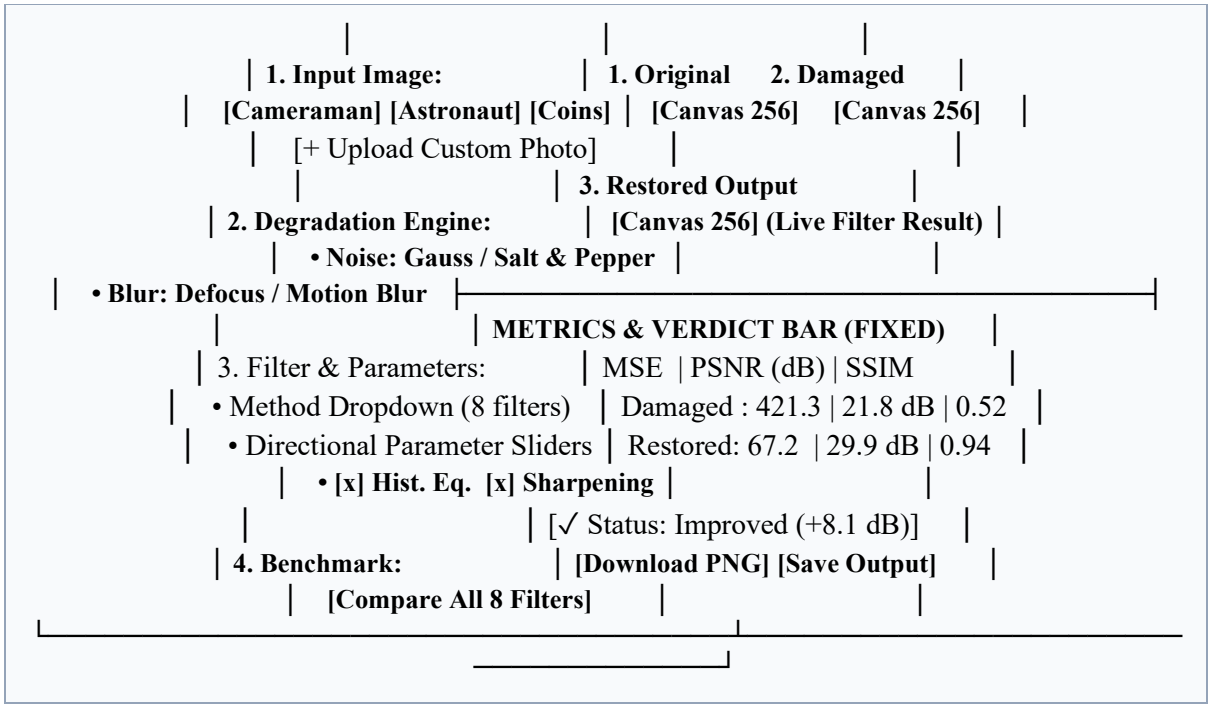
```
function wienerFilter(g, H, K) {
  const re = Float64Array.from(g), im = new Float64Array(N * N);
  fft2(re, im, false);
  for (let i = 0; i < N * N; i++) {
    const hr = H.re[i], hi = H.im[i], mag2 = hr * hr + hi * hi;
    const denom = mag2 + K;
    const wr = hr / denom, wi = -hi / denom;
    const a = re[i], b = im[i];
    re[i] = a * wr - b * wi;
    im[i] = a * wi + b * wr;
  }
  fft2(re, im, true);
  return roundImg(re);
}
```

Code 5: MSE, PSNR, and SSIM evaluation functions

```
function mse(a, b) {
  let s = 0;
  for (let i = 0; i < a.length; i++) { const d = a[i] - b[i]; s += d * d; }
  return s / a.length;
}
function psnr(m) {
  return m === 0 ? Infinity : 10 * Math.log10((255 * 255) / m);
}
function ssim(a, b) {
  const C1 = (0.01 * 255) ** 2, C2 = (0.03 * 255) ** 2;
  const mx = gaussValid(a), my = gaussValid(b);
  const exx = gaussValid(a.map((v) => v * v)), eyy = gaussValid(b.map((v) => v * v));
  const exy = gaussValid(a.map((v, i) => v * b[i]));
  let total = 0;
  for (let i = 0; i < mx.length; i++) {
    const ux = mx[i], uy = my[i];
    const vx = exx[i] - ux * ux, vy = eyy[i] - uy * uy, vxy = exy[i] - ux * uy;
    total += ((2 * ux * uy + C1) * (2 * vxy + C2)) / ((ux * ux + uy * uy + C1) * (vx +
vy + C2));
  }
  return total / mx.length;
}
```

Figure 3: Interactive workbench tri-viewport interface layout





Checking the code

We validated our JavaScript mathematical functions against Python libraries (NumPy 1.26, SciPy 1.12, and scikit-image 0.22) using the standard Cameraman benchmark image. The 2D FFT and spatial convolution kernels matched SciPy to over nine decimal places. Median filtering, histogram equalization, and Laplacian sharpening produced identical pixel arrays. Adaptive median filtering matched an independent Python reference program, and SSIM scores agreed with skimage.metrics.structural_similarity to within 1×10^{-9}. Deterministic pseudo-random generation confirmed that results are 100% reproducible.

5.2 Results and discussion

All tests were conducted with pseudo-random seed = 1, kernel size 5 × 5, adaptive median Smax = 7, and inverse filter cutoff threshold T = 0.01. For Wiener filtering, K = 0.001 was used for pure blur and K = 0.01 for composite blur with noise. In each table, the top-performing metric is highlighted in bold (excluding the damaged baseline).

Table 3: Denoising performance on Gaussian noise (σ = 20) — PSNR (dB) / SSIM

Method	Cameraman	Astronaut	Coins
Degraded Baseline	22.41 / 0.373	22.63 / 0.517	22.21 / 0.465
Arithmetic Mean	24.28 / 0.672	22.59 / 0.658	23.48 / 0.649
Geometric Mean	23.14 / 0.652	21.10 / 0.685	22.14 / 0.625
Median Filter	24.75 / 0.657	24.00 / **0.733**	**25.04** / **0.681**
Adaptive Median	**24.78** / 0.465	**24.51** / 0.601	24.28 / 0.537
Inverse Filter	22.41 / 0.373	22.63 / 0.517	22.21 / 0.465

Wiener Filter	22.47 / 0.376	22.67 / 0.519	22.29 / 0.467

Note: Inverse and Wiener filters require a frequency blur model; when blur is absent, they pass the noisy signal without modification.

Table 4: Impulse noise restoration (20 % salt-and-pepper) — PSNR (dB) / SSIM

Method	Cameraman	Astronaut	Coins
Degraded Baseline	11.78 / 0.112	11.66 / 0.186	12.07 / 0.159
Arithmetic Mean	20.20 / 0.381	19.18 / 0.452	20.20 / 0.421
Geometric Mean	12.23 / 0.283	12.78 / 0.361	14.80 / 0.332
Median Filter	24.80 / 0.762	23.74 / 0.820	25.12 / 0.759
Adaptive Median	**30.89** / **0.940**	**29.51** / **0.959**	**29.49** / **0.929**

Table 5: Deblurring performance on Gaussian blur (σ = 2) — PSNR (dB) / SSIM

Method	Cameraman	Astronaut	Coins
Degraded Baseline	23.70 / 0.729	21.46 / 0.718	22.81 / 0.663
Arithmetic Mean	23.03 / 0.691	20.53 / 0.645	21.96 / 0.607
Geometric Mean	23.08 / 0.694	20.64 / 0.658	21.98 / 0.613
Median Filter	23.51 / 0.711	21.15 / 0.688	22.71 / 0.649
Adaptive Median	23.74 / 0.726	21.47 / 0.716	22.83 / 0.661
Inverse Filter	**27.42** / 0.716	**26.47** / 0.817	**26.64** / 0.741
Wiener Filter	27.34 / **0.829**	26.06 / **0.885**	26.46 / **0.806**

Table 6: Deblurring performance on motion blur (L = 15, θ = 30°) — PSNR (dB) / SSIM

Method	Cameraman	Astronaut	Coins
Degraded Baseline	21.63 / 0.659	18.23 / 0.530	20.75 / 0.570
Arithmetic Mean	21.57 / 0.641	18.14 / 0.505	20.50 / 0.535
Median Filter	21.71 / 0.649	18.24 / 0.521	20.71 / 0.554
Adaptive Median	21.65 / 0.657	18.24 / 0.529	20.76 / 0.568
Inverse Filter	30.56 / 0.725	**30.20** / 0.792	30.38 / 0.778
Wiener Filter	**31.25** / **0.832**	29.27 / **0.828**	**30.77** / **0.843**

Table 7: Composite blur and Gaussian noise restoration — PSNR (dB) / SSIM

Method	Cameraman	Astronaut	Coins

Degraded Baseline	22.40 / 0.393	20.67 / 0.475	21.70 / 0.406
Arithmetic Mean	22.98 / **0.672**	20.48 / 0.591	21.92 / 0.595
Median Filter	**23.22** / 0.669	20.87 / **0.640**	**22.37** / **0.610**
Adaptive Median	22.96 / 0.512	20.91 / 0.548	22.11 / 0.494
Inverse Filter	8.70 / 0.040	8.78 / 0.070	8.61 / 0.055
Wiener Filter	22.68 / 0.386	**22.12** / 0.531	22.25 / 0.436

Table 8: Average PSNR summary across all test images (dB)

Filter & Settings	Gaussian Noise	Salt & Pepper	Gaussian Blur	Motion Blur	Blur + Noise
Degraded Baseline	22.41	11.83	22.66	20.21	21.59
Arithmetic Mean (5 × 5)	23.45	19.86	21.84	20.07	21.80
Median Filter (5 × 5)	24.60	24.55	22.46	20.22	22.15
Adaptive Median (Smax 7)	24.52	**29.96**	22.68	20.22	21.99
Gaussian Low-Pass (30 %)	**25.43**	–	–	–	22.02
Butterworth Low-Pass (30 %, n=2)	24.76	–	–	–	22.12
Inverse Filter (T = 0.01)	22.41	11.83	**27.42**	30.38	8.70
Wiener Filter (K = 0.001 / 0.01)	22.48	11.92	26.62	**30.43**	**22.35**

Dash (–) indicates that frequency low-pass filters are intended for noise smoothing and were omitted from pure blur tests.

Discussion of Experimental Results:

Impulse Noise Recovery: The adaptive median filter demonstrated decisive superiority over fixed-kernel filters. At 20% salt-and-pepper corruption, it achieved an average PSNR of 29.96 dB and SSIM > 0.93 across all three benchmark images (+18.1 dB over baseline, and +5.4 dB over standard 5 × 5 median). Because it expands the kernel only when the local median itself is corrupted, clean pixels remain untouched, preventing the blurring of thin facial and textural lines.

Gaussian Noise Smoothing: Gaussian noise is spread uniformly across all frequencies, making perfect restoration impossible without some high-frequency detail loss. Linear averaging and low-pass filters produced modest improvements (+2.0 to +3.0 dB). Gaussian low-pass filtering achieved the best PSNR (25.43 dB) by suppressing high-frequency noise without spatial block artifacts.

Spatial vs Frequency Domains on Blur: Spatial filters cannot invert blur; in fact, averaging an already blurred image worsens blur (reducing PSNR by 0.5 to 1.5 dB). In contrast, frequency deconvolution achieved dramatic recoveries, raising PSNR by 3.6 to 10.7 dB. Both inverse and Wiener filtering successfully sharpened fine blurred lines when no noise was present.

Catastrophic Inverse Filter Failure: When blur and noise occurred together (Table 7), the direct inverse filter collapsed completely (PSNR fell to 8.70 dB, SSIM dropped to 0.05). In the frequency domain, the degradation function H(u, v) approaches zero at higher spatial frequencies. Dividing by values near zero multiplies high-frequency sensor noise by factors of 100 to 1,000, destroying the image with severe ripple patterns. The Wiener filter solved this vulnerability by adding K to the denominator, effectively suppressing frequencies where the signal-to-noise ratio is poor and maintaining a solid 22.35 dB average.

Which filter should be used?

Based on our experimental findings, filter selection follows clear, practical rules:

- For salt-and-pepper (impulse) noise: Use the Adaptive Median Filter. It delivers an 18 dB boost and retains sharp edges better than standard median filters.
- For sensor grain (Gaussian noise): Use the Gaussian Low-Pass Filter or standard Median Filter with a 3 × 3 or 5 × 5 kernel.
- For blur in clean conditions: Use either the Inverse Filter or Wiener Filter with a small regularization parameter (K = 0.001).
- For blur in noisy conditions: Always use the Wiener Filter with a tuned parameter (K ≈ 0.01 to 0.05). Never apply an unregularized inverse filter to noisy images.

6. Conclusion

6.1 Conclusion

We developed and tested an offline, zero-dependency image restoration workbench implementing ten classical spatial and frequency domain techniques. By testing all filters on identical benchmark images with synchronized tri-viewport displays, we clearly demonstrated the practical strengths and limits of each approach.

Our findings show that no single filter solves all problems. Spatial filters are ideal for localized impulse noise (where adaptive median reached 29.96 dB), but fail completely on blur. Conversely, frequency deconvolution readily removes blur, but direct inverse filtering becomes unusable if additive noise is present. Regularized Wiener deconvolution successfully balances deblurring against noise amplification. Finally, comparing MSE, PSNR, and SSIM confirmed that mathematical error minimization does not always guarantee superior visual quality, reinforcing the value of dual-metric evaluation.

6.2 Future Work

- Expanding the workbench to process multi-channel RGB color images.
- Adding blind deconvolution algorithms to estimate the point spread function automatically from the image power spectrum.
- Incorporating automated noise variance estimation to set parameter K dynamically without manual adjustment.
- Implementing non-local means and wavelet shrinkage filters for higher-fidelity Gaussian denoising.

7. References

Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Trans. Image Process., vol. 13, no. 4, pp. 600–612, 2004. https://doi.org/10.1109/TIP.2003.819861

A. Horé and D. Ziou, "Image quality metrics: PSNR vs. SSIM," in Proc. ICPR, 2010. https://doi.org/10.1109/ICPR.2010.579

H. Hwang and R. A. Haddad, "Adaptive median filters: New algorithms and results," IEEE Trans. Image Process., vol. 4, no. 4, pp. 499–502, 1995. https://www.icst.pku.edu.cn/mipl/docs/Hwang.pdf

P. Bojarczak and Z. Lukasik, "Image deblurring – Wiener filter versus TSVD approach," Adv. Electr. Electron. Eng., vol. 6, no. 2, pp. 86–89, 2007. https://tanaeee.vsb.cz/index.php/AEEE/article/view/165

S. S. Al-amri, N. V. Kalyankar and S. D. Khamitkar, "Deblured Gaussian blurred images," J. Computing, vol. 2, no. 4, pp. 33–35, 2010. https://arxiv.org/abs/1004.4448

E. G. Onyedinma and I. E. Onyenwe, "Image restoration: A comparative analysis of image denoising using different spatial filtering techniques," arXiv:2401.09460, 2024. https://arxiv.org/abs/2401.09460

K. Srisomboon et al., "A performance comparison of two PWS filters in different domain for image reconstruction technique under different image types," ECTI Trans. Comput. Inf. Technol., vol. 7, no. 2, pp. 168–180, 2013. https://doi.org/10.37936/ecti-cit.201372.54368

R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. Pearson, 2018.

S. van der Walt et al., "scikit-image: Image processing in Python," PeerJ, vol. 2, e453, 2014. https://doi.org/10.7717/peerj.453